

부산대학교
PUSAN NATIONAL UNIVERSITY

Graph Representations

A Matrix or a List

Storing Edges and What It Costs

DS3600082-140
자료구조
2026학년도 2학기

오용경
데이터사이언스전공
yongkyung.oh@pusan.ac.kr

Section 1 • Graph Representations

Adjacency Matrix

모든 쌍을 한 칸씩 두는 표현을 살펴본다

간선을 행렬이나 리스트에 저장한다

정점과 간선을 정했으니 이제 그것을 저장할 방법을 정해야 한다

방법	표현
모든 쌍에 자리를 만든다	인접행렬
이어진 것만 적어 둔다	인접리스트

인접행렬

정의

인접행렬(adjacency matrix)

$n \times n$ 크기의 표를 두고 i에서 j로 가는 간선이 있으면 그 칸을 참으로 두는 표현이다.
n은 정점의 수다. 간선이 있든 없든 모든 칸이 자리를 차지한다.

i행 j열 한 칸은 i에서 j로 직접 가는 간선의 존재를 나타낸다

정점 12개의 인접행렬

12개 정점이므로 $12 \times 12 = 144$칸이 필요하다

	0	1	2	3	4	5	6	7	8	9	10	11
0	0	**1**	0	0	**1**	0	0	0	0	0	0	0
1	**1**	0	**1**	0	0	**1**	**1**	0	0	0	0	0
2	0	**1**	0	**1**	0	0	**1**	0	0	0	0	0
3	0	0	**1**	0	0	0	0	**1**	0	0	0	0
4	**1**	0	0	0	0	**1**	0	0	**1**	0	0	0
5	0	**1**	**1**	0	**1**	0	**1**	0	0	**1**	0	0
6	0	0	**1**	0	0	**1**	0	**1**	0	0	**1**	0
7	0	0	0	**1**	0	0	**1**	0	0	0	0	**1**
8	0	0	0	0	**1**	0	0	0	0	**1**	0	0
9	0	0	0	0	0	**1**	0	0	**1**	0	**1**	0
10	0	0	0	0	0	0	**1**	0	0	**1**	0	**1**
11	0	0	0	0	0	0	0	**1**	0	0	**1**	0

*i*행 *j*열의 1은 *i*에서 *j*로 가는 간선이 있다는 뜻이다. 144칸 가운데 1은 36개뿐이다.

Adapted from Morin (2013), Open Data Structures, Figure 12.2.

인접행렬의 연산

연산	비용	하는 일
간선 있는지 확인	$O(1)$	한 칸을 읽는다
간선 추가·삭제	$O(1)$	한 칸을 바꾼다
이웃 전부 보기	$O(n)$	한 행을 훑는다

> 인접행렬은 두 정점 사이에 간선이 있는지 $O(1)$에 확인한다

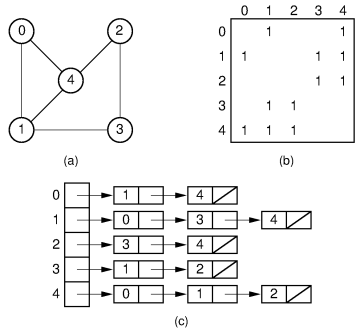
이웃을 찾는 데 $O(n)$이 드는 이유

근거	내용
이웃이 적힌 곳	한 행의 참인 칸에 흩어져 있다
찾는 방법	그 행의 n칸을 모두 확인한다
이웃이 2개뿐일 때	그래도 n칸을 다 본다

Caution 이웃이 적어도 한 행을 모두 확인하고 없는 간선에도 자리를 쓰므로 비용은 줄지 않는다

같은 그래프를 두 방법으로

정점 다섯 개짜리 그래프를 행렬과 리스트로 나란히 적어 본다

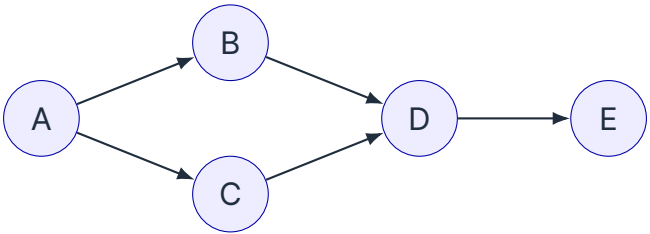


(a) 무향 그래프(간선 6개), (b) 대각선 대칭인 인접행렬, (c) 인접리스트. (c)는 각 정점의 목록을 연결리스트로 담았다.

Source: OpenDSA (n.d.), Shaffer et al.

다섯 정점의 간선 다섯 개

이번에는 방향이 있는 다섯 정점 그래프에서 같은 일을 한다



A에서는 B와 C로, D에서는 E로 간선이 나간다. D로 들어오는 간선은 두 개다

한 행과 한 열을 읽으면

행은 나가는 간선, 열은 들어오는 간선을 보여 준다

	A	B	C	D	E
A	0	1	1	0	0
B	0	0	0	1	0
C	0	0	0	1	0
D	0	0	0	0	1
E	0	0	0	0	0

1인 칸은 다섯 개다. 인접행렬에는 간선이 없는 칸도 함께 저장한다

행과 열에서 간선을 찾는다

질문	읽을 자리
B에서 D로 가는 간선이 있는가	B행 D열 한 칸은 1
A에서 어디로 간선이 나가는가	A행 다섯 칸에서 B, C
D로 어디서 간선이 들어오는가	D열 다섯 칸에서 B, C

간선 하나는 한 칸, 나가거나 들어오는 간선 전체는 다섯 칸을 확인한다

무향 그래프의 인접행렬

항목	내용
담는 방법	(i, j)와 (j, i)를 함께 참으로 둔다
행렬의 모양	대각선을 기준으로 대칭이다

> 앞의 (b)처럼 대칭이므로 절반만 저장할 수도 있지만 공간은 여전히 $O(n^2)$이다

인접행렬의 칸은 간선 여부나 가중치를 담는다

담는 값	말할 수 있는 것	쓰는 곳
참·거짓	간선이 있는가	가중치 없는 그래프
수치	간선에 붙은 가중치	가중치 그래프

> 같은 표에 무엇을 담느냐에 따라 답할 수 있는 질문이 달라진다

행렬이 자리를 쓰는 방식

간선이 하나도 없어도 n^2칸은 그대로 필요하다

간선 수	쓰는 칸	채워진 비율
0개	144	0%
36개	144	25%
132개	144	92%

132는 자기 루프 자리인 대각선 12칸을 뺀 최대 간선 수 12×11이다

Checkpoint

Adjacency Matrix

- 인접행렬의 한 칸은 무엇을 뜻하는가?
- 간선 하나를 확인하는 비용과 이웃 전부를 보는 비용은 각각 얼마인가?
- 무향 그래프에서 인접행렬은 어떤 모양이 되는가?

Section 2 • Graph Representations

Adjacency List

이어진 것만 적어 두는 표현을 살펴본다

인접리스트

정의

인접리스트(adjacency list)

각 정점에 직접 인접한 정점들의 목록을 저장하는 표현이다.
정점별 목록과 실제 간선 항목을 저장하므로 필요한 공간은 $O(n+m)$이다. n은 정점 수, m은 간선 수다.

목록을 연결리스트, 배열, `set` 어느 것으로 담아도 이 표현의 이름은 인접리스트다

같은 그래프의 인접리스트

앞의 행렬과 같은 그래프이며 목록에 적힌 이웃은 36개뿐이다

0	1	2	3	4	5	6	7	8	9	10	11
1 4	0 2 6 5	1 3 6	2 7	0 5 8	1 2 6 9 4	5 2 7 10	6 3 11	4 9	8 5 10	9 6 11	10 7

맨 윗줄이 정점 번호이고, 그 아래가 각 정점에서 간선 하나로 바로 갈 수 있는 정점의 목록이다.

Source: Morin (2013), Open Data Structures, Figure 12.3.

같은 다섯 간선을 목록으로 읽는다

한 정점의 나가는 간선만 그 정점의 목록에 적는다

정점	이웃 목록	간선 확인
A	B, C	A→D 확인은 B, C와 차례로 비교
B	D	B→D 확인은 D 한 번 비교
C	D	D로 들어오는 간선을 찾을 때 확인
D	E	D에서 나가는 간선은 E 하나
E	비어 있음	나가는 간선이 없다

D로 들어오는 간선은 한 목록에 모이지 않아 다섯 목록을 확인해야 한다

인접리스트의 연산

연산	비용	하는 일
간선 있는지 확인	차수만큼	배열 목록을 훑는다
간선 추가, 중복 미확인	$O(1)$	목록 끝에 붙인다
간선 추가, 중복 차단	차수만큼	확인한 뒤 붙인다
이웃 전부 보기	차수만큼	목록을 그대로 준다

이웃을 `set`이나 `dict`에 담으면 간선 확인과 중복 차단도 기대 $O(1)$이다

들어오는 간선을 찾으려면

간선	찾는 방법과 비용
나가는 간선	그 정점의 목록에 있다
들어오는 간선	모든 목록을 다 봐야 하므로 $O(n+m)$

Caution 인접리스트는 나가는 방향만 바로 주므로 양쪽이 필요하면 두 종류의 목록을 둔다

정점 이름을 배열 인덱스나 해시 키로 찾는다

정점의 이름	목록을 담는 곳	찾는 비용
0부터 $n-1$까지	배열	$O(1)$
임의의 값	`dict`	기대 $O(1)$

정점 이름을 `dict`의 키로 쓰면 해당 인접 목록을 찾을 수 있다

Checkpoint

Adjacency List

- 인접리스트는 무엇을 저장하는가?
- 인접리스트와 연결리스트는 어떻게 다른가?
- 들어오는 간선을 찾는 비용이 큰 이유는 무엇인가?

Section 3 • Graph Representations

Degree, Sparse and Dense

간선이 얼마나 많은지를 세는 말을 정한다

차수

정의

차수(degree)

한 정점에 연결된 간선의 수다.
유향 그래프에서는 나가는 간선 수를 진출 차수(out-degree)라고 한다.
들어오는 간선 수는 진입 차수(in-degree)라고 하여 둘을 구분한다.

인접리스트에서 한 정점의 목록 길이가 곧 그 정점의 진출 차수다

간선 하나는 두 합에 한 번씩 들어간다

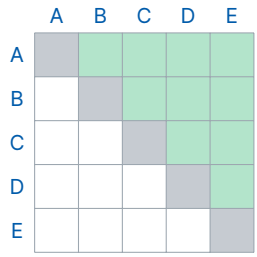
앞의 다섯 간선을 출발 정점과 도착 정점에서 각각 센다

정점	진출 차수	진입 차수
A	2	0
B	1	1
C	1	1
D	1	2
E	0	1
합	5	5

각 간선은 출발 정점의 진출 차수와 도착 정점의 진입 차수에 한 번씩 더해진다

자기 루프가 없으면 유향 간선은 최대 $n(n-1)$개다

5×5 행렬에서 간선이 들어갈 수 있는 칸을 센다



회색 대각선 5칸은 자기 루프 자리, 초록 10칸은 대각선 위쪽 절반이다.

그래프	최대 간선 수	$n=5$	$n=12$
유향	$n^2-n=n(n-1)$	20	132
무향	$n(n-1)/2$	10	66

무향 간선 하나는 (i,j)와 (j,i) 두 칸을 함께 쓰므로 위쪽 절반만 센다.

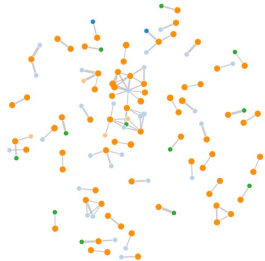
앞의 144칸 가운데 대각선 12칸을 빼면 유향 간선은 최대 132개다

희소 그래프와 밀집 그래프

정의

희소 그래프(sparse graph)

가능한 간선 가운데 실제로 있는 것이 아주 적은 그래프이다.
간선이 n^2에 가까울 만큼 많으면
밀집 그래프(dense graph)라고 한다.



실제 협업 그래프의 한 모습이다. 정점 대부분에 간선이 한두 개뿐이다.

희소와 밀집은 n이 커질 때 m이 느는 빠르기로 가르므로 작은 예 하나로는 정할 수 없다

Source: UBC DSCI 221 (n.d.), Graphs: Collaboration Graph.

공간의 차이

항목	인접행렬	인접리스트
공간	n^2	$n + m$
$n = 12$, $m = 36$	144	48
n이 1000이면	1,000,000	$1000 + m$

차수를 세는 비용

차수	인접행렬	인접리스트
진출 차수	$O(n)$ 한 행을 센다	목록 길이, 길이를 저장하면 $O(1)$
진입 차수	$O(n)$ 한 열을 센다	모든 목록을 본다

인접행렬은 한 행과 한 열이 각각 n칸이므로 두 차수를 모두 $O(n)$에 센다

Checkpoint

Degree, Sparse and Dense

- 유향 그래프에서 진출 차수를 모두 더하면 왜 간선 수 m이 되는가?
- 자기 루프가 없을 때 정점 n개의 유향 간선은 최대 몇 개인가?
- 어떤 그래프를 희소 그래프라고 하는가?

Section 4 • Graph Representations

Choosing a Representation

연산과 그래프의 성질을 보고 표현을 고른다

연산별 비용

같은 연산도 두 표현에서 비용이 다르다

연산	인접행렬	인접리스트
간선 확인	$O(1)$	배열은 차수만큼, `set`은 기대 $O(1)$
간선 추가, 중복 차단	$O(1)$	배열은 차수만큼, `set`은 기대 $O(1)$
이웃 전부 보기	$O(n)$	차수만큼
들어오는 간선 찾기	$O(n)$	$O(n+m)$
공간	n^2	$n+m$

사전 두 단계로 담은 인접리스트

NetworkX 그래프 `G`는 정점에서 이웃 사전으로 가는 두 단계의 사전이다

질문	접근	비용을 정하는 것
정점 `u` 찾기	바깥 사전 `G[u]`	해시 조회
`u`의 이웃 보기	안쪽 사전의 키 순회	`u`의 차수
간선 속성 읽기	`G[u][v]`	해시 조회 두 번

> 인접리스트는 기존 사전과 순회를 조합해 그래프 연산을 구현한다

Adapted from Downey (2021), `graph.ipynb`, "Graph Representation."

간선 목록을 입력받아 인접리스트를 만든다

정점은 0부터 n-1까지다. set으로 중복을 막고 정렬된 이웃 목록을 반환한다

```python
def build_adjacency(n, edges, directed=False):
    if n < 0:
        raise ValueError("n < 0")
    adj = [set() for _ in range(n)]
    for u, v in edges:
        if not (0 <= u < n and 0 <= v < n):
            raise ValueError("vertex out of range")
        adj[u].add(v)
        if not directed:
            adj[v].add(u)
    return [sorted(vs) for vs in adj]
```

```
n = 4
edges = [(2, 0), (2, 0),
         (1, 2)]
```

정점	반환 목록
0	[2]
1	[2]
2	[0, 1]
3	[]

기본값은 무향이므로 양쪽에 넣는다. directed=True이면 출발 쪽에만 넣는다. 고립 정점 3의 빈 목록도 남는다.

표현을 고를 때 볼 것

고려할 것	판단 기준
필요한 공간	n과 m의 관계
자주 하는 연산	간선 확인 또는 이웃 열거
이웃을 담는 컨테이너	배열, `set`, 다른 컨테이너

정점 수가 변하면 고정 크기 인접행렬은 다시 만든다

표현	정점이 늘어날 때
인접행렬	크기가 고정된 2차원 배열이면 행렬 전체를 다시 만들어야 한다
인접리스트	목록 하나를 더하면 된다

> 정점 수가 변하는 그래프에서는 인접행렬의 재할당이 비싸다

Checkpoint

Choosing a Representation

- 간선 하나를 확인하는 비용은 두 표현에서 어떻게 다른가?
- 이웃을 자주 묻는다면 어느 표현이 유리한가?
- 정점이 늘면 고정 크기 인접행렬은 어떻게 바뀌는가?

How Do We Explore a Graph?

Next Lecture: Graph Traversal

- BFS가 층으로 넓히는 방식
- DFS가 한 갈래로 내려가는 방식
- 큐와 스택의 차이

오용경 yongkyung.oh@pusan.ac.kr